vmctl — Application Requirements

A vm-bhyve management app: terminal UI (TUI) and desktop GUI, with a built-in ease-of-use assistant.

Version 0.1 · Draft for design review · Target platform: FreeBSD (vm-bhyve v1.7.5), portable to Linux/macOS backends.

1. Overview & Goals

vmctl is a first-class management front-end for **vm-bhyve**, designed to feel as approachable as VirtualBox while exposing the full power of bhyve. It ships in two front-ends that share one core: a **TUI** (ratatui, runs in any terminal, including over SSH) and a **Desktop GUI** (native windowed app). Both are driven by a shared Rust core and a plain-English **assistant** that turns intent into safe vm(8) operations.

Design goals, in priority order:

- **Powerful** — cover the full vm-bhyve lifecycle: create, install, start/stop, snapshot/clone, networking, storage, console.
- **Easy to use** — VirtualBox-style flows: a creation wizard, one-click actions, sensible defaults, no config-file editing required for common tasks.
- **Safe** — destructive actions (destroy, discard, rollback, poweroff) are confirmed and, where possible, reversible via snapshot.
- **Honest** — surface real system state and real errors; never pretend an action succeeded.
- **Portable core** — the vm-bhyve backend is one implementation behind a trait, so a future VirtualBox/QEMU backend can slot in.

2. Users & Use Cases

Persona	Need	Primary surface
Desktop user	Run VMs without learning bhyve; create a VM from an ISO in a few clicks.	Desktop GUI + assistant
Sysadmin / power user	Manage many VMs over SSH; script and automate; full control.	TUI + CLI fast-path
Developer	Spin up disposable test VMs, snapshot/rollback, iterate quickly.	TUI + Desktop
Newcomer	Doesn't know vm(8) syntax; asks in plain English.	Assistant (both)

3. Architecture (shared core)

One core crate (**vmctl-core**) owns all logic; the TUI and Desktop are thin views over it. This guarantees the two front-ends never diverge in behavior.

Layer	Responsibility	Notes
vmctl-core	Typed vm(8) wrapper, VM model, actions, state machine, assistant engine.	Already scaffolded: vmbhyve.rs, actions.rs
Backend trait	Abstract VM ops (list/create/start/stop/…). vm-bhyve is the first impl.	Future: VirtualBox, QEMU, cloud
Assistant engine	Plain-English → intent → validated action plan → confirm → execute.	Shared by TUI + Desktop
TUI (ratatui)	Terminal panes: list, wizard, config, console, logs.	ca0e750: list + actions done
Desktop GUI	Windowed app: same views as native widgets + drag/drop + system tray.	e.g. egui / Tauri / Iced

4. Functional Requirements

4.1 VM Lifecycle & Inventory

ID	Requirement	Priority
F-101	List all VMs with name, state, vCPUs, memory, datastore, and console info (parses multi-word states like 'Locked (user)').	Must
F-102	Create a VM via wizard: template → datastore → size → memory → vCPUs → image/ISO → name (maps to `vm create`).	Must
F-103	Install from ISO (`vm install `) with a guided flow and progress.	Must
F-104	Start / stop / restart / suspend / reset / poweroff single or multiple VMs.	Must
F-105	Start-all / stop-all / suspend-all bulk operations.	Should
F-106	Show detailed VM info (`vm info`) in a structured panel.	Must
F-107	Rename, clone, and discard/delete VMs (with confirmation).	Should
F-108	Attach to VM console (`vm console`) — surfaced as a safe action; in TUI offered as a shell hand-off, in Desktop as an embedded terminal.	Must

4.2 Snapshots & Images

ID	Requirement	Priority
F-201	Create, list, rollback, and delete snapshots (`vm snapshot/rollback`).	Should
F-202	Clone from a snapshot (`vm clone name@snap newname`).	Should
F-203	Manage images: list, create, provision, destroy (`vm image …`).	Could
F-204	Download/list available ISOs and images (`vm iso`, `vm img`).	Could

4.3 Networking & Storage

ID	Requirement	Priority
F-301	Manage virtual switches: list, create (standard/standard-nat/private), edit, destroy (`vm switch …`).	Should
F-302	Manage datastores: list, add, remove (`vm datastore …`).	Should
F-303	Add devices to a VM: NIC, disk, custom (`vm add -t/-s`).	Should
F-304	Edit raw VM config in a safe editor with validation (`vm configure/edit`).	Could

4.4 Assistant (ease-of-use layer)

ID	Requirement	Priority
F-401	Accept plain-English intent (e.g. 'make a 4GB Ubuntu VM with 2 CPUs') and propose a concrete action plan.	Must
F-402	Show the plan (the exact vm(8) operations) and require confirmation before executing anything.	Must
F-403	Suggest a sensible default action for the selected VM (stopped → Start, running → Console/Stop).	Must
F-404	Explain errors and system state in plain language (e.g. non-root permission, missing ISO).	Should
F-405	Answer 'how do I …?' questions with the exact steps/commands.	Should

5. Non-Functional Requirements

ID	Category	Requirement
N-101	Safety	Destructive ops require explicit confirmation; never auto-run on selection. Assistant always shows the plan first.
N-102	Safety	Privilege handling: detect non-root and surface a clear, actionable error rather than a crash.
N-103	Reliability	Terminal state is always restored on exit (no stuck raw mode). GUI never blocks the main thread on vm(8) calls.
N-104	Performance	VM list refresh < 200ms for typical fleets; async/background execution of long ops (install, start).
N-105	Portability	Core is OS-agnostic behind the backend trait; TUI works over SSH; Desktop targets FreeBSD/Linux/macOS.
N-106	Testability	All parsing and action-mapping logic unit-tested; backend trait mockable for UI tests.
N-107	Accessibility	TUI: full keyboard control, no mouse required. Desktop: keyboard shortcuts, screen-reader-friendly labels.
N-108	Honesty	UI reflects real vm(8) exit status; failed actions show the real stderr, never a false success.

6. UX Requirements (VirtualBox-style ease of use)

- **Creation wizard** — a step-through flow (Template → Resources → Storage → Network → Summary → Create) with recommended defaults pre-filled.
- **One-click actions** — a prominent action bar / toolbar; the most likely action is highlighted.
- **At-a-glance state** — color-coded VM states (running/stopped/locked/suspended) in both views.
- **Contextual help** — the assistant is always one keystroke/click away and knows the current selection.
- **Progress & feedback** — long operations (install, clone) show progress; all actions give clear success/failure feedback.
- **Undo where possible** — snapshot-before-destructive-op as an opt-in safety net.

7. Scope & Milestones

Milestone	Deliverable	Status
M0 — Core foundation	Typed vm(8) wrapper + list parser + action layer.	Done (0c18a73, ca0e750)
M1 — TUI MVP	VM list pane, action bar, create wizard, console hand-off.	In progress
M2 — Assistant	Plain-English → plan → confirm → execute, in TUI.	Planned
M3 — Desktop GUI	Windowed app over the same core: list, wizard, actions, tray.	Planned
M4 — Snapshots/net/storage	Snapshot/clone, switch & datastore management in both views.	Planned

Out of scope for v1: live migration, clustering/federation, multi-host orchestration, and non-bhyve backends (the backend trait is designed to allow them later).